

Reviewer Pack — Moment Matrix Eigenvalue Decay and SOS Degree for Max-CUT

Entry 1ca152a211e8 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 21:37:18 UTC

Moment Matrix Eigenvalue Decay and SOS Degree for Max-CUT

Entry ID: 1ca152a211e8 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 21:37:18 UTC

1. Conjecture

Field A (mathematical branch): Real Algebraic Geometry **Field B** (complexity object): SOS Degree for Max-CUT

Statement:

For a random Max-CUT instance on n variables, the minimal eigenvalue of the degree- d moment matrix of its SOS relaxation decays as $\Theta(1/d)$ with constant depending on n . For all $n \leq 40$ and $d \geq 2$, the absolute value of the minimal eigenvalue satisfies $|\lambda_{\min}| \geq c(n)/d$ where $c(n) = \Theta(n^\alpha)$ for $\alpha \in (0, 1)$.

Rationale (proposer's reasoning):

The eigenvalues of the moment matrix encode the geometry of the feasible region for the SOS relaxation. Their decay rate with degree d reveals how the relaxation's tightness scales, linking algebraic structure to optimization complexity. This connects real algebraic geometry's spectral properties to SOS hierarchy behavior.

Taxonomy category: META_COMPLEXITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 30077bb800294b26

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_max_cut_instance(n):
    return [random.choice([0, 1]) for _ in range(n)]

def degree_d_moment_matrix(instance, d):
    n = len(instance)
    M = [[0] * (d + 1) for _ in range(d + 1)]
    for i in range(n):
        if instance[i] == 1:
            for j in range(i + 1, n):
                if instance[j] == 1:
                    for k in range(d + 1):
                        M[k][k] += math.comb(k + 2, 2)
    return M

def gaussian_elimination(A_b):
    n = len(A_b)
    for i in range(n):
        max_row = i
        for j in range(i + 1, n):
            if abs(A_b[j][i]) > abs(A_b[max_row][i]):
                max_row = j
        A_b[i], A_b[max_row] = A_b[max_row], A_b[i]
        factor = A_b[i][i]
        for j in range(i, n + 1):
            A_b[i][j] /= factor
        for j in range(n):
            if j != i:
                factor = A_b[j][i]
                for k in range(i, n + 1):
                    A_b[j][k] -= factor * A_b[i][k]
    return A_b

def solve_linear_system(A, b):
    A_b = [A[i] + [b[i]] for i in range(len(A))]
```

```

A_b = gaussian_elimination(A_b)
x = [0] * len(b)
for i in range(len(x) - 1, -1, -1):
    x[i] = A_b[i][-1]
    for j in range(i + 1, len(x)):
        x[i] -= A_b[i][j] * x[j]
return x

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    instance = generate_max_cut_instance(n)

    results = []
    for d in [2, 4, 8]:
        M = degree_d_moment_matrix(instance, d)
        try:
            eigenvalues = solve_linear_system(M, [0] * (n + 1))
            lambda_min = min(abs(eigenvalue) for eigenvalue in eigenvalues)
            results.append((d, lambda_min))
        except ZeroDivisionError:
            return {
                "metric_name": "min_eigenvalue",
                "metric_value": None,
                "instances_tested": 0,
                "conjecture_holds": False,
                "counterexample": "division_by_zero"
            }

    if not results:
        return {
            "metric_name": "min_eigenvalue",
            "metric_value": None,
            "instances_tested": 0,
            "conjecture_holds": False,
            "counterexample": "no_results"
        }

    lambda_mins = [result[1] for result in results]
    d_values = [result[0] for result in results]

    # Polynomial interpolation to estimate c(n)
    def interpolate(x, y):
        n = len(x)
        A = [[x[i]**j for j in range(n)] + [y[i]] for i in range(n)]
        solution = solve_linear_system(A, [1] * n)
        return lambda t: sum(solution[j] * t**j for j in range(n))

    c_n_func = interpolate(d_values, lambda_mins)
    c_n = c_n_func(1) # Estimate c(n) for d=1

    if c_n is None or c_n <= 0:
        return {
            "metric_name": "min_eigenvalue",
            "metric_value": None,
            "instances_tested": len(results),

```

```

        "conjecture_holds": False,
        "counterexample": "invalid_c_n"
    }

    # Check if the decay rate matches  $\Theta(1/d)$  with  $n$ -dependent constants
    decay_rates = [abs(lambda_min * d - c_n / d) for lambda_min, d in results]
    max_decay_rate = max(decay_rates)

    return {
        "metric_name": "min_eigenvalue",
        "metric_value": max_decay_rate,
        "instances_tested": len(results),
        "conjecture_holds": max_decay_rate <= 1e-6, # Adjust threshold as needed
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 35)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results) / len(results))
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    else:
        for r in results:
            if not r["conjecture_holds"]:
                counterexample = f"d={r['instances_tested'][-1][0]}, lambda_min={r['instances_tested'][-1][1]}, c_n={r['instances_tested'][-1][2]}"
                print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={seed}")
                break
        else:
            print("RESULT: INCONCLUSIVE")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6c2f9359.py", line 130, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6c2f9359.py", line 69, in run_trial

```

```
eigenvalues = solve_linear_system(M, [0] * (n + 1))
```

IndexError: list index out of range

Critic verdict: CHALLENGE

skipped: test crashed before producing data

Verdict: INCONCLUSIVE

Test crashed with `IndexError` before producing results, preventing validation of the conjecture. | next: Debug the `IndexError` in `solve_linear_system` to ensure proper matrix dimensions for all $n \leq 40$ and $d \geq 2$

Total LLM calls: 8

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 326471 ms total latency. Provider mix: {'ollama remote': 8}

12. Reproducibility

To reproduce this cycle:

Sandbox file: research/pvsnp sandbox/test *.py (per-cycle)